

Unit 2

Control Structure

2.1 Conditional Statements in C

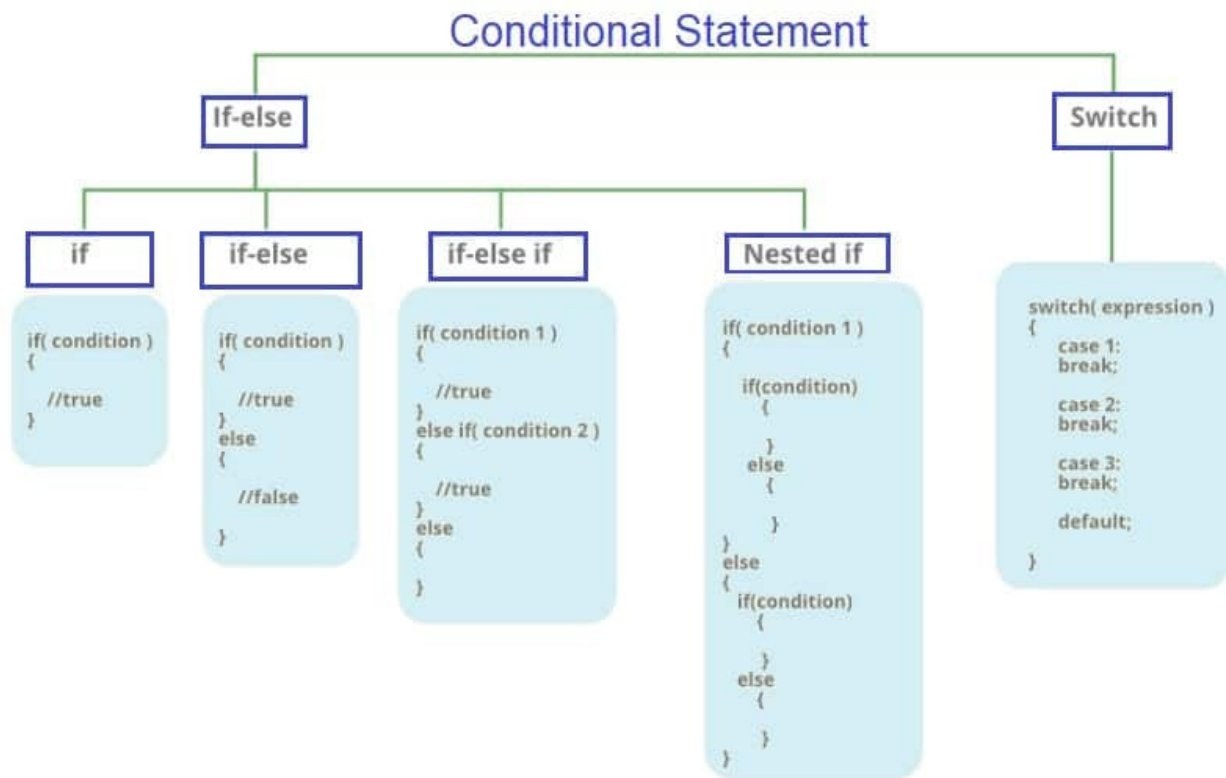
The **conditional statements** (also known as decision control structures) such as if, if else, switch, etc. are used for decision-making purposes in C programs.

They are also known as Decision-Making Statements and are used to evaluate one or more conditions and make the decision whether to execute a set of statements or not. These decision-making statements in programming languages decide the direction of the flow of program execution.

Need of Conditional Statements

There come situations in real life when we need to make some decisions and based on these decisions, we decide what we should do next. Similar situations arise in programming also where we need to make some decisions and based on these decisions we will execute the next block of code. For example, in C if x occurs then execute y else execute z. There can also be multiple conditions like in C if x occurs then execute p, else if condition y occurs execute q, else execute r. This condition of C else-if is one of the many ways of importing multiple conditions.

Types of Conditional Statements in C



Following are the decision-making statements available in C:

1. **if Statement**
2. **if-else Statement**
3. **Nested if Statement**
4. **if-else-if Ladder**
5. **switch Statement**

1. if in C

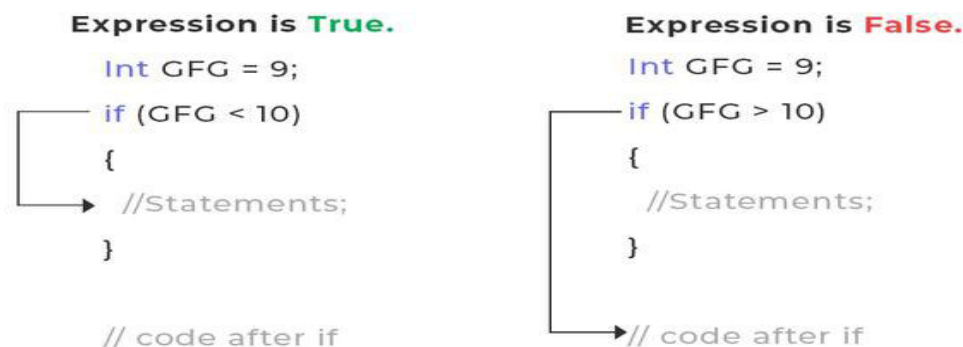
The if statement is the most simple decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e if a certain condition is true then a block of statements is executed otherwise not.

Syntax of if Statement

```
if(condition)
{
    // Statements to execute if
    // condition is true
}
```

Here, the **condition** after evaluation will be either true or false. C if statement accepts boolean values – if the value is true then it will execute the block of statements below it otherwise not. If we do not provide the curly braces '{' and '}' after if(condition) then by default if statement will consider the first immediately below statement to be inside its block.

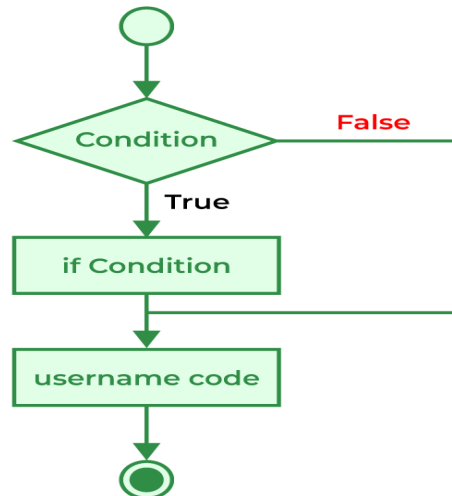
How if in C works?



The working of the 'if' statement in C is as follows:

1. **STEP 1:** When the program control comes to the 'if' statement, the test expression is evaluated.
2. **STEP 2A:** If the condition is true, the statements inside the 'if' block are executed.
3. **STEP 2B:** If the expression is false, the statements inside the 'if' body are not executed.
4. **STEP 3:** Program control moves out of the 'if' block and the code after the if block is executed.

Flowchart of if Statement



Example of if in C

// C program to illustrate If statement

```
#include <stdio.h>
```

```
int main()
{
    int i = 10;

    if (i > 15) {
        printf("10 is greater than 15");
    }

    printf("I am Not in if");
}
```

Output

I am Not in if

As the condition present in the 'if' statement is false. So, the block below the if statement is not executed.

Advantages of if Statement in C

- It is the simplest decision-making statement.
- It is easy to use and understand.
- It can evaluate expressions of all types such as int, char, bool, etc.

Disadvantages of if Statement in C

- It contains only a single block. In case when there are multiply related if blocks, all the blocks will be tested even when the matching if block is found at the start
- When there are a large number of expressions, the code of the if block gets complex and unreadable.
- It is slower for a large number of conditions.

2. if-else in C

The *if* statement alone tells us that if a condition is true it will execute a block of statements and if the condition is false it won't. But what if we want to do something else when the condition is false? Here comes the C *else* statement. We can use the *else* statement with the *if* statement to execute a block of code when the condition is false. The if-else statement consists of two blocks, one for false expression and one for true expression.

Syntax of if else in C

```
if (condition)
{
    // Executes this block if
    // condition is true
}
else
{
    // Executes this block if
    // condition is false
}
```

How if-else Statement works?

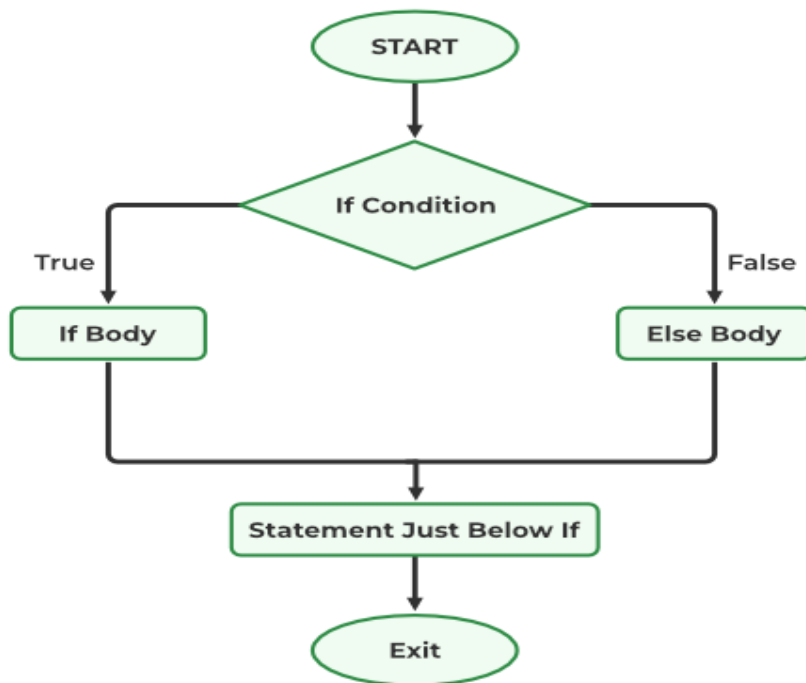
Working of the if-else statement in C is explained below:

1. When the program control first comes to the if-else block, the test condition is checked.
2. If the test condition is **true**:
 - The if block is executed.
3. If the test condition is **false**:
 - The else block is executed
4. After that, the program control continues to the statements below the if-else statement.

```
if (condition)
{
    // execute code statements of if block when
    // condition is true
}
else
{
    // execute code statements of if block when
    // condition is true
}
```

Note: Any non-zero and non-null values are assumed to be true, and zero or null values are assumed to be false.

Flowchart of if-else Statement



Example of if else in C

1. C program to illustrate If else statement

```
#include <stdio.h>
```

```
int main()
{
    int i = 20;

    if (i < 15) {

        printf("i is smaller than 15");
    }
    else {

        printf("i is greater than 15");
    }
    return 0;
}
```

Output

i is greater than 15

The block of code following the *else* statement is executed as the condition present in the *if* statement is false.

2.C Program to Demonstrate the working of if-else statement

```
#include <stdio.h>
```

```
int main()
{
    // Some random number
    int num = 9911234;

    // checking the condition at the start of if block
    if (num % 2 == 0) {
        // executed when the number is even
        printf("Number is even");
    }
    // else block
    else {
        // executed when the number is odd
        printf("Number is Odd");
    }

    return 0;
}
```

Output

Number is even

3. C Program to check whether the person is eligible to vote or not

```
#include <stdio.h>
```

```
int main()
{
    // declaring age of two person
    int p1_age = 15;
    int p2_age = 25;

    // checking eligibility of person 1
    if (p1_age < 18)
        printf("Person 1 is not eligible to vote.\n");
    else
        printf("Person 1 is eligible to vote.\n");

    // checking eligiblity of person 2
    if (p2_age < 18)
        printf("Person 2 is not eligible to vote.\n");
    else
        printf("Person 2 is eligible to vote.");

    return 0;
}
```

Output

Person 1 is not eligible to vote.

Person 2 is eligible to vote.

Advantages of if-else Statement

- The if-else statement enables the user to execute different statements based on different conditions.
- It can evaluate test expressions of type int, char, boolean, and more.
- It helps in modifying the flow of the program.
- It is simple, efficient, and easier to read when there is less number of conditions.

Disadvantages of if-else Statement

- If there are a lot of if statements present, the code becomes unreadable and complex.
- It also becomes slower compared to the switch statement.

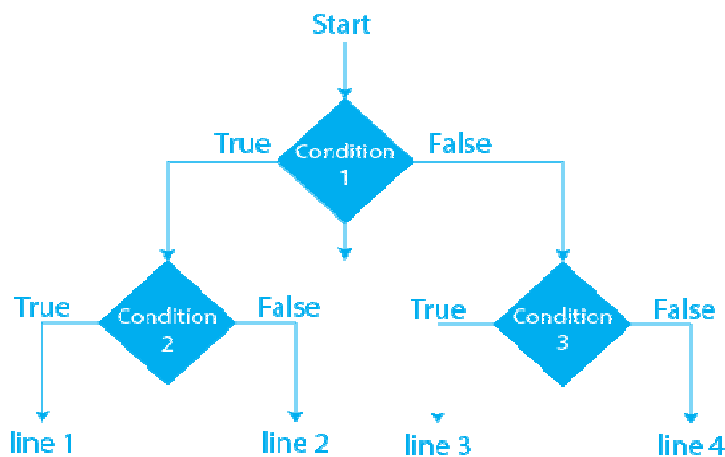
3. Nested if-else in C

A nested if in C is an if statement that is the target of another if statement. Nested if statements mean an if statement inside another if statement. Yes, C allow us to nested if statements within if statements, i.e, we can place an if statement inside another if statement.

Syntax of Nested if-else

```
if (condition1)
{
    // Executes when condition1 is true
    if (condition2)
    {
        // Executes when condition2 is true
    }
    else
    {
        // Executes when condition2 is false
    }
}
```

Flowchart of Nested if-else



Flow Chart

Example of Nested if-else

1. C program to illustrate nested-if statement

```
#include <stdio.h>
```

```
int main()
{
    int i = 10;

    if (i == 10) {
        // First if statement
        if (i < 15)
            printf("i is smaller than 15\n");

        // Nested - if statement
        // Will only be executed if statement above
        // is true
        if (i < 12)
            printf("i is smaller than 12 too\n");
        else
            printf("i is greater than 15");
    }

    return 0;
}
```

Output

i is smaller than 15

i is smaller than 12 too

2. write a program that takes in a number and checks whether it is *positive, negative, or zero*.

```
#include <stdio.h>
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    if (num > 0) {
        printf("%d is positive.\n", num);
    } else {
        if (num < 0) {
            printf("%d is negative.\n", num);
        } else {
            printf("%d is zero.\n", num);
        }
    }
    return 0;
}
```


Output:

Enter a number: 10
10 is positive.

Enter a number: -5
-5 is negative.

Enter a number: 0
0 is zero.

4. if-else-if Ladder in C

The if else if statements are used when the user has to decide among multiple options. The C if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the C else-if ladder is bypassed. If none of the conditions is true, then the final else statement will be executed. if-else-if ladder is similar to the switch statement.

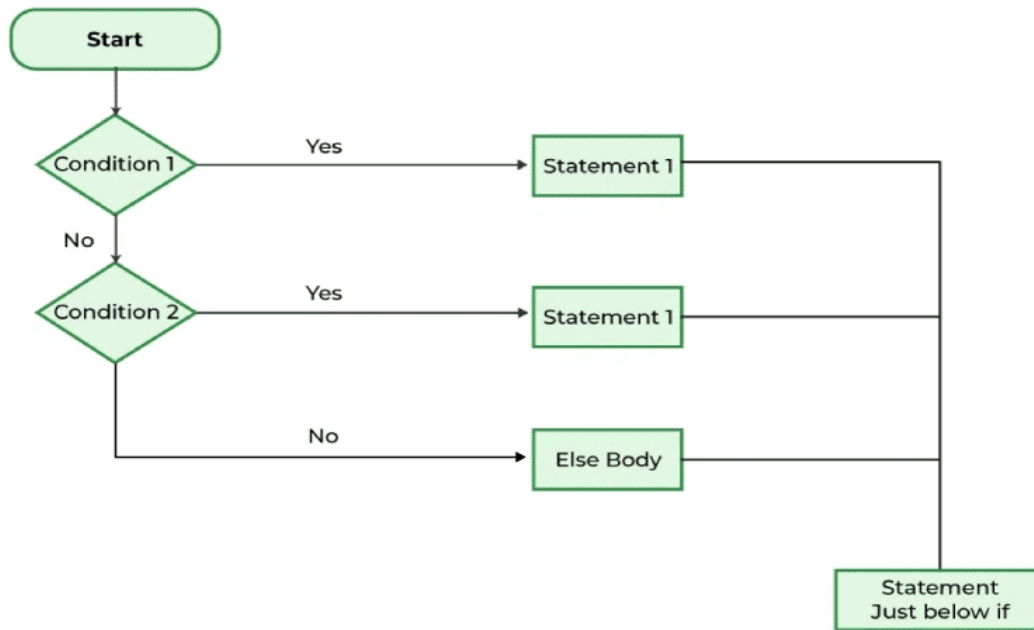
Syntax of if-else-if Ladder

```
if (condition)
    statement;
else if (condition)
    statement;
.
.
else
    statement;
```

Working Flow of the if-else-if ladder:

1. The flow of the program falls into the if block.
2. The flow jumps to 1st Condition
3. 1st Condition is tested respectively:
 - If the following Condition yields true, go to Step 4.
 - If the following Condition yields false, go to Step 5.
4. The present block is executed. Goto Step 7.
5. The flow jumps to Condition 2.
 - If the following Condition yields true, go to step 4.
 - If the following Condition yields false, go to Step 6.
6. The flow jumps to Condition 3.
 - If the following Condition yields true, go to step 4.
 - If the following Condition yields false, execute the else block. Goto Step 7.
7. Exits the if-else-if ladder.

Flowchart of if-else-if Ladder



Note: A if-else ladder can exclude else block.

Example of if-else-if Ladder

1. C program to illustrate nested-if statement

```
#include <stdio.h>
```

```
int main()
{
    int i = 20;

    if (i == 10)
        printf("i is 10");
    else if (i == 15)
        printf("i is 15");
    else if (i == 20)
        printf("i is 20");
    else
        printf("i is not present");
}
```

Output

i is 20

2. Check whether a number is positive, negative or 0

// C Program to check whether a number is positive, negative or 0 using if else if ladder

```
#include <stdio.h>
```

```
int main()
{
```

```

    int n = 0;
    // all Positive numbers will make this
    // condition true
    if (n > 0) {
        printf("Positive");
    }

    // all Negative numbers will make this
    // condition true
    else if (n < 0) {
        printf("Negative");
    }

    // if a number is neither Positive nor Negative
    else {
        printf("Zero");
    }
    return 0;
}

```

Output

Zero

3. Calculate Grade According to marks

```

// C Program to Calculate Grade According to marks
// using the if else if ladder
#include <stdio.h>
int main()
{
    int marks = 91;
    if (marks <= 100 && marks >= 90)
        printf("A+ Grade");
    else if (marks < 90 && marks >= 80)
        printf("A Grade");
    else if (marks < 80 && marks >= 70)
        printf("B Grade");
    else if (marks < 70 && marks >= 60)
        printf("C Grade");
    else if (marks < 60 && marks >= 50)
        printf("D Grade");
    else
        printf("F Failed");
    return 0;
}

```

Output

A+ Grade

5. switch Statement in C

- The switch case statement is an alternative to the if else if ladder that can be used to execute the conditional code based on the value of the variable specified in the switch statement. The switch block consists of cases to be executed based on the value of the switch variable.
- Switch case statement evaluates a given expression and based on the evaluated value(matching a certain condition), it executes the statements associated with it. Basically, it is used to perform different actions based on different conditions(cases).
- Switch case statements follow a selection-control mechanism and allow a value to change control of execution.
- They are a substitute for long if statements that compare a variable to several integral values.
- The switch statement is a multiway branch statement. It provides an easy way to dispatch execution to different parts of code based on the value of the expression.
- In C, the switch case statement is used for executing one condition from multiple conditions. It is similar to an if-else-if ladder.
- The switch statement consists of conditional-based cases and a default case.

Syntax of switch statement

```
switch (expression) {  
    case value1:  
        statements;  
    case value2:  
        statements;  
    ....  
    ....  
    ....  
    default:  
        statements;  
}
```

Note: The switch expression should evaluate to either integer or character. It cannot evaluate any other data type.

Rules of the switch case statement

Following are some of the rules that we need to follow while using the switch statement:

1. In a switch statement, the “**case value**” must be of “**char**” and “**int**” type.
2. There can be one or N number of cases.
3. The values in the case must be **unique**.
4. Each statement of the case can have a break statement. It is optional.
5. The default Statement is also optional.

How switch Statement Work?

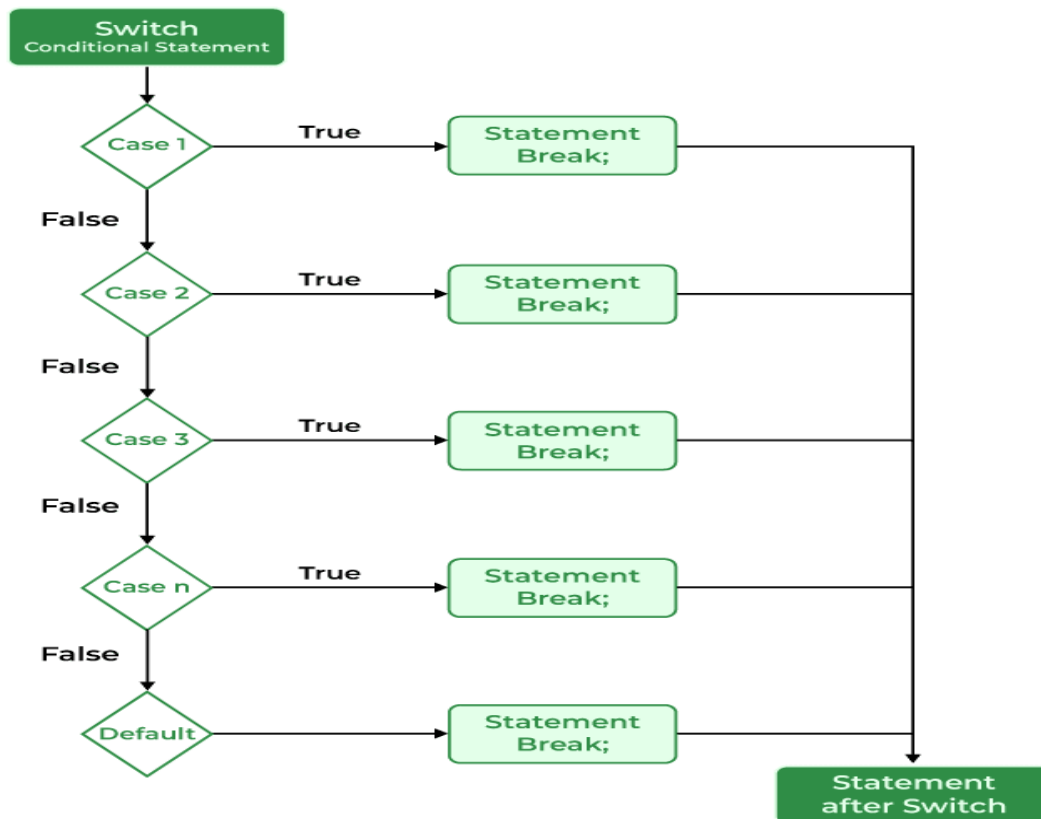
The working of the switch statement in C is as follows:

1. **Step 1:** The switch variable is evaluated.
2. **Step 2:** The evaluated value is matched against all the present cases.

3. **Step 3A:** If the matching case value is found, the associated code is executed.
4. **Step 3B:** If the matching code is not found, then the default case is executed if present.
5. **Step 4A:** If the break keyword is present in the case, then program control breaks out of the switch statement.
6. **Step 4B:** If the break keyword is not present, then all the cases after the matching case are executed.
7. **Step 5:** Statements after the switch statement are executed.

We can also understand the working of the switch statement in C using the flowchart.

Flowchart of switch



Example of switch Statement

1. C Program to illustrate the use of switch statement

```

#include <stdio.h>
int main()
{
    // variable to be used in switch statement
    int var = 2;

    // declaring switch cases
    switch (var) {
    case 1:
        printf("Case 1 is executed");
        break;
    case 2:

```

```
        break;
default:
    printf("Default Case is executed");
    break;
}

return 0;
}
```

Output

Case 2 is executed

2. C Program to print the day of the week using a switch case.

// C program to print the day using switch
#include <stdio.h>

```
// Driver Code
int main()
{
    int day = 2;

    printf("The day with number %d is ", day);
    switch (day) {
        case 1:
            printf("Monday");
            break;
        case 2:
            printf("Tuesday");
            break;
        case 3:
            printf("Wednesday");
            break;
        case 4:
            printf("Thursday");
            break;
        case 5:
            printf("Friday");
            break;
        case 6:
            printf("Saturday");
            break;
        case 7:
            printf("Sunday");
            break;
        default:
            printf("Invalid Input");
            break;
    }
    return 0;
}
```

Output

The day with number 2 is Tuesday

3. Simple Calculator using switch case in C

// C Program to create a simple calculator using switch

// statement

#include <stdio.h>

#include <stdlib.h>

// driver code

int main()

{

 // switch variable

 char choice;

 // operands

 int x, y;

 while (1) {

 printf("Enter the Operator (+,-,*,./)\nEnter x to "

 "exit\n");

 scanf(" %c", &choice);

 // for exit

 if (choice == 'x') {

 exit(0);

 }

 printf("Enter the two numbers: ");

 scanf("%d %d", &x, &y);

 // switch case with operation for each operator

 switch (choice) {

 case '+':

 printf("%d + %d = %d\n", x, y, x + y);

 break;

 case '-':

 printf("%d - %d = %d\n", x, y, x - y);

 break;

 case '*':

 printf("%d * %d = %d\n", x, y, x * y);

 break;

 case '/':

 printf("%d / %d = %d\n", x, y, x / y);

 break;

 default:

 printf("Invalid Operator Input\n");

 }

 }

 return 0;

}

Output

Enter the operator (+, -, *, /)

Enter x to exit

+

Enter the two numbers: 100 + 200

100 + 200 = 300

Advantages of C switch Statement

1. Easier to read than if else if.
2. Easier to debug and maintain for a large number of conditions.
3. Faster execution speed.

Disadvantages of C switch Statement

1. Switch case can only evaluate int or char type.
2. No support for logical expressions.
3. Have to keep in mind to add a break in every case.

2.2 Looping Statements in C

Loops in programming are used to repeat a block of code until the specified condition is met. A loop statement allows programmers to execute a statement or group of statements multiple times without repetition of code.

C program to illustrate need of loops

```
#include <stdio.h>
int main()
{
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");
    printf( "Hello World\n");

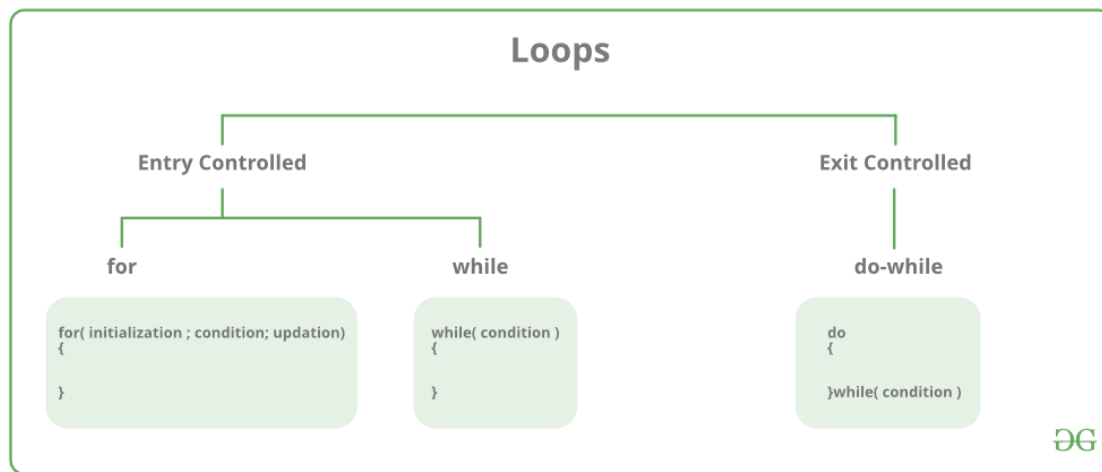
    return 0;
}
```

Output

```
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
```

There are mainly two types of loops in C Programming:

1. **Entry Controlled loops:** In Entry controlled loops the test condition is checked before entering the main body of the loop. **For Loop and While Loop** is Entry-controlled loops.
2. **Exit Controlled loops:** In Exit controlled loops the test condition is evaluated at the end of the loop body. The loop body will execute at least once, irrespective of whether the condition is true or false. **do-while Loop** is Exit Controlled loop.



Loop Type	Description
for loop	first Initializes, then condition check, then executes the body and at last, the update is done.
while loop	first Initializes, then condition checks, and then executes the body, and updating can be inside the body.
do-while loop	do-while first executes the body and then the condition check is done.

1. for Loop in C

- In C programming, loops are responsible for performing repetitive tasks using a short code block that executes until the condition holds true. In this article, we will learn about **for loop in C**.
- The **for loop** in C Language provides a functionality/feature to repeat a set of statements a defined number of times. The for loop is in itself a form of an **entry-controlled loop**.
- Unlike the while loop and do...while loop, the for loop contains the initialization, condition, and updating statements as part of its syntax. It is mainly used to traverse arrays, vectors, and other data structures.

Syntax of for Loop

```
for(initialization; check/test expression; updatation)
{
    // body consisting of multiple statements
}
```

Structure of for Loop

The for loop follows a very structured approach where it begins with initializing a condition then checks the condition and in the end executes conditional statements followed by an updation of values.

1. **Initialization:** This step initializes a loop control variable with an initial value that helps to progress the loop or helps in checking the condition. It acts as the index value when iterating an array or string.
2. **Check/Test Condition:** This step of the **for loop** defines the condition that determines whether the loop should continue executing or not. The condition is checked before each iteration and if it is true then the iteration of the loop continues otherwise the loop is terminated.
3. **Body:** It is the set of statements i.e. variables, functions, etc that is executed repeatedly till the condition is true. It is enclosed within curly braces { }.
4. **Updation:** This specifies how the loop control variable should be updated after each iteration of the loop. Generally, it is the incrementation (variable++) or decrementation (variable --) of the loop control variable.

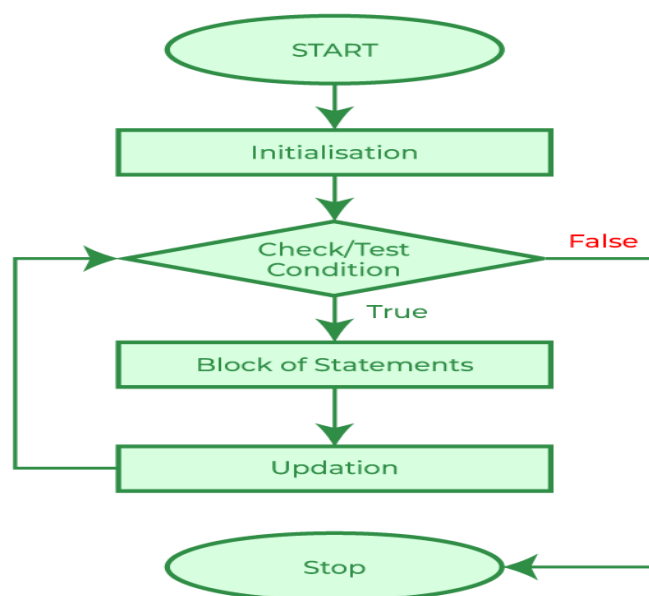
How for Loop Works?

The working of for loop is mentioned below:

- **Step 1:** Initialization is the basic step of for loop this step occurs only once during the start of the loop. During Initialization, variables are declared, or already existing variables are assigned some value.
- **Step 2:** During the Second Step condition statements are checked and only if the condition is the satisfied loop we can further process otherwise loop is broken.
- **Step 3:** All the statements inside the loop are executed.
- **Step 4:** Updating the values of variables has been done as defined in the loop.
Continue to Step 2 till the loop breaks.

Flowchart of for Loop

The following program illustrates how to use for loop in C:



Example of for loop

The following program illustrates how to use for loop in C:

1. C program to demonstrate for loop

```
#include <stdio.h>
int main()
{
    int a = 0;

    // 'a' <= 5 is the check/test expression
    // The loop will function if and only if 'a' is less
    // than 5
    // 'a++' will increments it's value by this so that the
    // loop can iterate for further evaluation

    // conditional statement
    for (a = 1; a <= 5; a++)
    {
        // statement will be printed
        printf("Hello World\n");
    }

    // Return statement to tell that everything executed
    return 0;
}
```

Output

Hello World
Hello World
Hello World
Hello World
Hello World

2. Simple program of for loop that prints table of 1.

```
#include<stdio.h>
int main(){
    int i=0;
    for(i=1;i<=10;i++){
        printf("%d \n",i);
    }
    return 0;
}
```

Output

1
2

3
4
5
6
7
8
9
10

3. Print table for the given number using C for loop

```
#include<stdio.h>
int main(){
int i=1,number=0;
printf("Enter a number: ");
scanf("%d",&number);
for(i=1;i<=10;i++){
printf("%d \n",(number*i));
}
return 0;
}
```

Output:

```
Enter a number: 2
2
4
6
8
10
12
14
16
18
20
```

Nested for loop in C

C provides the feature of a nested loop where we can place a loop inside another loop.

Syntax

```
for( .. ; .. ; .. ){
    for( .. ; .. ; .. ){
        ....
    }
}
```

Infinitive for loop in C

To make a for loop infinite, we need not give any expression in the syntax. Instead of that, we need to provide two semicolons to validate the syntax of the for loop. This will work as an infinite for loop.

```
#include<stdio.h>

void main ()
{
    for(;;)
    {
        printf("welcome to javatpoint");
    }
}
```

Advantages of for Loop

- Provides code reusability
- Code size decreases
- Traversing in data structures like array and string becomes easy.

Disadvantages of for Loop

- Can't skip any element while traversing
- Only a single condition is followed

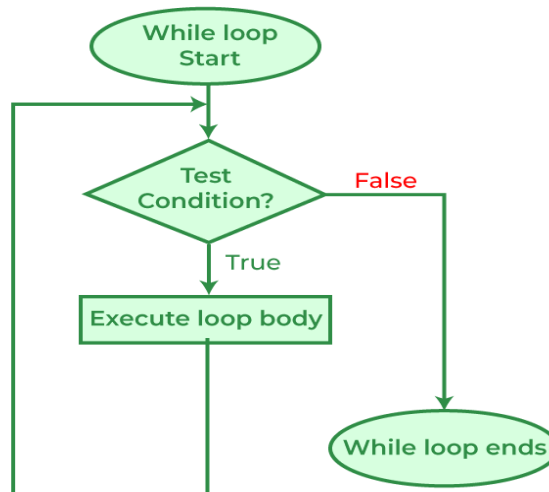
2. while loop in C

- The **while Loop** is an entry-controlled loop in C programming language. This loop can be used to iterate a part of code while the given condition remains true.
- While loop does not depends upon the number of iterations. In the for loop the number of iterations was previously known to us but in while loop the execution is terminated on the basics of the test condition.
- If the test condition will become false then it will break from the while loop else body will be executed.

Syntax:

```
initialization_expression;
while (test_expression)
{
    // body of the while loop
    update_expression;
}
```

Flow Diagram for while loop:



Examples of while loop

// C program to illustrate while loop

```
#include <stdio.h>
int main()
{
    // Initialization expression
    int i = 2;

    // Test expression
    while(i < 10)
    {
        // loop body
        printf( "Hello World\n");

        // update expression
        i++;
    }

    return 0;
}
```

Output:

```
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
Hello World
```

2. C program to demonstrate while loop

```
#include <stdio.h>
int main()
{
    // Initialization of loop variable
    int i = 0;

    // setting test expression as (i < 5), means the loop will execute till i is less than 5
    while (i < 5) {

        // loop statements
        printf("CWIT");

        // updating the loop variable
        i++;
    }
    return 0;
}
```

Output

```
CWIT
CWIT
CWIT
CWIT
CWIT
```

3. do-while Loop

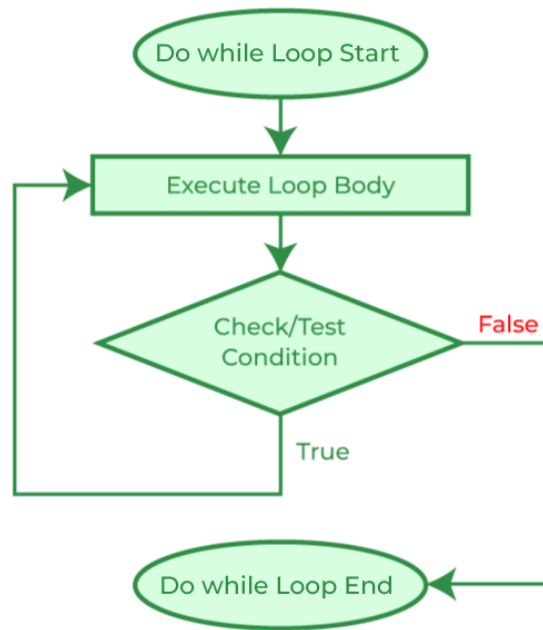
- The do-while loop is similar to a while loop but the only difference lies in the do-while loop test condition which is tested at the end of the body.
- The do while in C is a loop statement used to repeat some part of the code till the given condition is fulfilled.
- It is a form of exit controlled loop where the test condition is checked after executing the body of the loop.
- In the do-while loop, the loop body will **execute at least once** irrespective of the test condition.
- It is mainly used to traverse array, vectors and other data structure.

Syntax:

```
initialization_expression;
do
{
    // body of do-while loop
    update_expression;

} while (test_expression);
```


Flowchart of do..while loop



Example of do..while loop

1. C program to illustrate do-while loop

```
#include <stdio.h>
int main()
{
    // Initialization expression
    int i = 2;

    do
    {
        // loop body
        printf( "Hello World\n");

        // Update expression
        i++;

        // Test expression
    } while (i < 1);

    return 0;
}
```

Output

Hello World

Above program will evaluate ($i < 1$) as false since $i = 2$. But still, as it is a do-while loop the body will be executed once.

2. Write a simple program that questions the user for a *password* and keeps asking until the right password is input.

```
#include <stdio.h>
#include <string.h>
int main() {
    char password[] = "secret";
    char input[20];
    do {
        printf("Enter the password: ");
        scanf("%s", input);
    } while (strcmp(input, password) != 0);
    printf("Access granted!\n");
    return 0;
}
```

Output:

```
Enter the password: 123
Enter the password: abc
Enter the password: secret
Access Granted!
```

The program runs as follows:

1. The following header files are included: **<stdio.h>** for standard *input* and *output* routines and **<string.h>** for string *manipulation functions*.
2. The correct password is defined as a *character array* (**char password[]**) with the value **"secret"**
3. After that, we define another character array input to store the user's input.
4. The **do keyword** indicates that the code block included within the **loop** will be performed at least once.
5. Using the **printf() function**, we display a prompt requesting the user to input their password inside the Loop.
6. Next, we read the *user's input* using the **scanf() function** and store it in the *input array*.
7. After reading the *input*, we use the **strcmp() function** to compare the input with the correct password. If the strings are *equal*, the **strcmp function** returns 0. So, we continue looping as long as the input and the password are not equal.
8. Once the *correct password* is entered, the loop terminates, and we print **"Access granted!"** using the **printf() function**.
9. After that, the program returns 0 to indicate successful execution.

2.3 Branching statements in C

These statements give programmers the freedom to jump to a location in their code under any circumstances.

Branching statements are mainly categorized as follows.

1. Conditional Branching Statements
2. Unconditional Branching Statements

1. Conditional Branching Statements:

In C, **conditional branching statements** are used to execute particular code blocks based on a condition (as needed). These branching instructions in C allow programmers to run the code only when specific conditions are met. The various categories of conditional branching statements in C are as follows:

- **if Statement**
- **if-else Statement**
- **nested if-else Statement**
- **switch Statement**

2. Unconditional Branching Statements:

C uses **unconditional branching statements** to alter the typical course of program execution. These statements give programmers the freedom to jump to a specific location in their code under any circumstance. The various categories of unconditional branching statements in C are as follows:

- **goto Statement**
- **break Statement**
- **continue Statement**

I. goto Statement:

- The **"goto"** command enables programmers to leap directly to a **designated labelled** statement within their code. It is an unconditional branching statement.
- The c goto statement is a jump statement which is sometimes also referred as an unconditional jump statement.
- The goto statement can be used to jump from anywhere to anywhere within a function.

Syntax of goto Statement in C:

Syntax1 | Syntax2

goto label; | label:

. | .

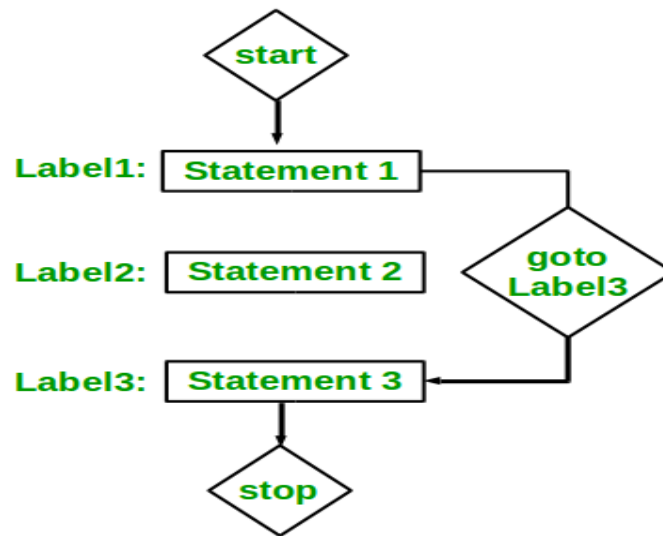
. | .

. | .

label: | goto label;

The **"goto"** statement executes the code in the block following a jump to the **"label"-labeled statement**. Any legal identifier can be used as the **"label"** after a colon (:).

Flow chart of goto statement



Examples of goto statements

1. Program to illustrate use of goto statement

```
#include <stdio.h>

int main() {
    int num = 1;
    if (num == 1) {
        goto label;
    }
    printf("This statement is skipped.\n");
    label:
    printf("The value of num is 1.\n");
    return 0;
}
```

Output:

The value of num is 1.

2. C program to check if a number is even or not using goto statement

```
#include <stdio.h>
// function to check even or not
void checkEvenOrNot(int num)
{
    if (num % 2 == 0)
        // jump to even
        goto even;
    else
        // jump to odd
        goto odd;

even:
    printf("%d is even", num);
    // return if even
    return;
odd:
    printf("%d is odd", num);
}

int main()
{
    int num = 26;
    checkEvenOrNot(num);
    return 0;
}
```

Output:

26 is even

3. C program to print numbers from 1 to 10 using goto statement

```
#include <stdio.h>
// function to print numbers from 1 to 10
void printNumbers()
{
    int n = 1;
label:
    printf("%d ", n);
    n++;
    if (n <= 10)
        goto label;
}

// Driver program to test above function
int main()
{
    printNumbers();
    return 0;
}
```

Output:

1 2 3 4 5 6 7 8 9 10

Disadvantages of Using goto Statement

- The use of the goto statement is highly discouraged as it makes the program logic very complex.
- The use of goto makes tracing the flow of the program very difficult.
- The use of goto makes the task of analyzing and verifying the correctness of programs (particularly those involving loops) very difficult.
- The use of goto can be simply avoided by using break and continue statements.

II. Break Statement in C

- The break statement is frequently employed in switch statement as well as looping construction like “for”, “while” and “do-while”.
- It enables you skip the statement that follows the loop or switch and end the execution of the closest enclosing loop or switch statement.
- The **break in C** is a loop control statement that breaks out of the loop when encountered. It can be used inside loops or switch statements to bring the control out of the block. The break statement can only break out of a single loop at a time.

Syntax of break in C

```
break;
```

Use of break in C

The break statement in C is used for breaking out of the loop. We can use it with any type of loop to bring the program control out of the loop.

Examples of break in C

C Program to use break Statement with Simple Loops

Break statements in C can be used with simple loops i.e, for loops, while loops, and do-while loops.

1. C Program to demonstrate break statement with for loop

```
#include <stdio.h>
int main()
{
    // using break inside for loop to terminate after 2 iteration
    printf("break in for loop\n");
    for (int i = 1; i < 5; i++) {
        if (i == 3) {
            break;
        }
        else {
            printf("%d ", i);
        }
    }

    // using break inside while loop to terminate after 2 iteration
    printf("\nbreak in while loop\n");
    int i = 1;
```

```

while (i < 20) {
    if (i == 3)
        break;
    else
        printf("%d ", i);
    i++;
}
return 0;
}

```

Output:

break in for loop

1 2

break in while loop

1 2

2. C Program to use break Statement with Nested Loops

Break statements can also be used when working with nested loops. The control will come out of only that loop in which the break statement is used.

// C program to illustrate using break statement in Nested loops

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // nested for loops with break statement at inner loop
```

```
    for (int i = 1; i <= 6; ++i) {
```

```
        for (int j = 1; j <= i; ++j) {
```

```
            if (i <= 4) {
```

```
                printf("%d ", j);
```

```
            }
```

```
        else {
```

```
            // if i > 4 then this innermost loop will
```

```
            // break
```

```
            break;
```

```
        }
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
return 0;
```

```
}
```

Output

1

1 2

1 2 3

1 2 3 4

Note: Break statement only breaks out of one loop at a time. So if in nested loop, we have used break in inner loop, the control will come to outer loop instead of breaking out of all the loops at once. We will have to use multiple break statements if we want to break out of all the loops.

III. continue Statement

In the C programming language, the **continue statement** is used to go to the next iteration of a loop while skipping the current iteration. Usually, this statement is used in loops like for or while.

Syntax of continue statement in C is as follows:

When a **loop encounters** the continue statement, the current iteration of the loop is ended, and control is returned to the **top** of the loop to begin the subsequent iteration.

Example:

Simple example to demonstrates how to use the C continues Statement.

```
#include <stdio.h>
int main() {
    int i;

    for (i = 0; i < 10; i++) {
        if (i % 2 == 0) {
            continue; // skip even numbers
        }

        printf("%d ", i);
    }

    return 0;
}
```

Output:

1 3 5 7 9

Explanation:

In the above example, the **for loop** iterates from **0** to **9**, and when the loop variable **i** is **even**, the continue statement is executed and control is transferred to the following iteration of the loop. As a result, the **printf statement** only prints **odd integers** and skips over **even values**.

Advantages of Branching Statements:

Better Decision Making: Branching statements give programmers the ability to decide what to do next in their code depending on certain circumstances. It improves the program's functionality and versatility by allowing it to respond and adapt to various conditions.

Readability of the code: *Branching statements* make the logic of the code clearer and simpler to understand. For other developers to understand and maintain the codebase, the conditional statements provide distinct signs of the program's decision-making process.

Code effectiveness: Branching statements optimize program execution by only running the necessary code blocks in accordance with the predetermined circumstances. It can make programs run more quickly and effectively, especially when employing the *"switch" statement* to process big sets of cases or complex decision trees.

Flexibility: Programming *flexibility* is provided through branching statements, which enables several code paths to be performed in response to diverse circumstances. This adaptability is especially helpful when the program needs to react quickly to altering inputs or outside influences.

Code Reusability: Branching statements make it easier to reuse code by enabling the execution of code blocks in response to various situations. Programmers can create modular code that can be easily reused in various portions of the program by utilizing branching statements efficiently.

Disadvantages of Branching Statements:

Code Complexity: When branching statements are overused or deeply nested, the code may become clearer and easier to comprehend. It may result in *maintenance problems, defects, and poorer-quality code*. It's crucial to utilize branching statements sparingly and aim for simplicity.

Readability Issues: Although branching statements can sometimes make code easier to comprehend, they can also cause issues if used carelessly. The code may become more difficult to read and understand if there are too many nested levels, complex conditional expressions, or duplicative sections of code.

Potential for Logical Mistakes: If the conditions are not adequately established and validated, branching statements run the danger of logical mistakes.

Code Maintenance: The intricacy of branching statements makes it more difficult to maintain the code. It can be difficult to comprehend the logic of the code and make the necessary adjustments without adding problems when there are numerous branching statements and nested conditions present.